TECH*LAND*

E-COMMERCE SEMESTER PROJECT – P1: TEAM, SETUP & PLANNING.

Team members:

Necsulea Andrei-Mihai : Backend & Database Lead
Onofrei Alexandru-Gabriel : Frontend & UI/UX Lead

Link repo:

https://github.com/necsulea-andrei/techland

Context, Niche & Target Audience

Store Niche:

A hybrid e-commerce platform bridging consumer IT hardware and industrial automation equipment.

Offers PC hardware (CPUs, RAM, storage, motherboards) alongside industrial automation modules (PLCs, digital/analog I/O expansion modules, sensors, DIN-rail power supplies).

Target Customers:

B2C (Individual Customers): Tech enthusiasts, students, and engineers looking for hardware upgrades (SSDs, RAM modules) or prototyping components for personal projects.

PDF

B2B (Enterprises & Production Facilities): Maintenance engineers, automation technicians, and small manufacturing businesses requiring quick sourcing of industrial PLCs and peripherals without lengthy offline quotation procedures.

Competitor Analysis (3 Real-World Stores)

1. PC Garage (Consumer IT Hardware Specialist)

What we will adopt: Intuitive category hierarchy and streamlined filtering by core technical specifications (storage capacity, memory speed, socket type).

What we will do differently: Expand beyond standard consumer hardware by integrating industrial automation components (PLCs and I/O modules) into a single, unified store catalog.

2. TME - Transfer Multisort Elektronik (Industrial & Electronics Distributor)

What we will adopt: Comprehensive technical parameter display (voltage ratings, input/output counts, industrial communication protocols).

What we will do differently: Replace complex, dated enterprise interfaces with a modern, clutter-free UI fully responsive on 360 px mobile screens.

3. RS Components (B2B Industrial Catalog)

What we will adopt: Direct unit purchasing with transparent real-time stock levels displayed per item.

What we will do differently: Eliminate heavy corporate onboarding forms; provide a streamlined self-service checkout process with integrated sandbox card payments and automated order status tracking.

Technology Stack

Frontend – Django Templates & Tailwind CSS:

Justification: Provides a fast, responsive user interface down to 360 px width while natively integrating with Django's built-in forms, CSRF protection, and session handling.

Backend – Django (Python):

Justification: Delivers a robust, production-ready framework with built-in authentication, ORM security, and streamlined business logic handling for our e-commerce workflows.

Database – PostgreSQL:

Justification: Ensures ACID-compliant transactional consistency and row-level locking mechanisms essential for concurrent stock management and order processing.

Payment Processor – Stripe (Test Mode):

Justification: Offers comprehensive sandbox support and reliable asynchronous webhooks to securely handle payment confirmation without trusting client-side redirects.

Week-by-Week Implementation Schedule (A1–A12)

Weeks 4–5: Setup, User Accounts & UI Foundations

Andrei (Backend & DB):

Set up PostgreSQL database schema, Docker configuration, and initial seed script (min. 30 products across 4 categories).

A4 (User accounts): Implement user registration, secure login with password hashing (bcrypt/Argon2), logout, and delivery address persistence.

Alexandru (Frontend & UI/UX):

Create full UI wireframes and base design system.

A11 (Responsive design - Core layout): Establish mobile-first styles, ensuring baseline viewports work seamlessly down to 360 px width.

Weeks 6–7: Catalog, Search & Milestone P2 Prototype

Andrei (Backend & DB):

A1 (Product catalog): Build paginated product API (max. 24 items/page), category filtering, and product detail data models.

A2 (Basic search): Implement server-side product name search with clear empty-state response handling.

Alexandru (Frontend & UI/UX):

A1 & A2 (Catalog & Search UI): Implement views for catalog, category pages, product details, and the search input bar.

Joint delivery: Deliver working prototype live demo (catalog loaded from seed data + authentication) and P2 specification document.

Weeks 8–9: Shopping Cart, Checkout & Stripe Integration

Andrei (Backend & DB):

A3 (Shopping cart): Handle server-side cart total calculations, stock rejection on overflow, and authenticated cart persistence after logout.

A6 (Sandbox payment): Integrate Stripe sandbox mode and secure asynchronous webhook handling to mark orders as *Paid*.

Alexandru (Frontend & UI/UX):

A3 (Cart UI): Build dynamic cart interface with item quantity controls and stock error notices.

A5 (Checkout): Implement multi-step checkout workflow: address selection, delivery method choice (at least 2 options), order summary, and order confirmation screen.

Weeks 10–11: Stock Concurrency, Admin Panel, Security & Automated Tests

Andrei (Backend & DB):

A7 (Stock management): Enforce atomic stock decrements upon payment confirmation, cancellation rollback, and race-condition prevention.

A10 (Basic security): Server-side form validation, CSRF protection, ORM parameterization, and strict server-side access control for admin routes.

A12 (Automated tests): Write a suite of at least 10 automated tests covering cart, totals, and payment confirmation, executable via a single command.

Alexandru (Frontend & UI/UX):

A8 (Order history): Build customer order history dashboard displaying dates, items, totals, and statuses.

A9 (Administration panel): Develop custom admin interface for product/category CRUD (with image upload) and order lifecycle management.

A11 (Responsive design - Final audit): Complete responsive pass across checkout, cart, and account screens at 360px width.

Top 3 Risks & Mitigation Strategies

1. Stock Concurrency & Race Conditions (Requirement A7)

Risk: Two users could attempt to purchase the last available unit simultaneously, causing an overselling race condition where stock drops below zero.

Mitigation: Use PostgreSQL atomic database transactions with row-level pessimistic locking (SELECT ... FOR UPDATE in Django ORM) during checkout validation and webhook order processing.

2. Payment Asynchrony & Local Webhook Debugging (Requirement A6)

Risk: Order status fails to transition to *Paid* if asynchronous Stripe webhooks are dropped, misconfigured, or unreachable on local development environments.

Mitigation: Use Stripe CLI (stripe listen --forward-to) locally for deterministic webhook delivery, verify Stripe cryptographic webhook signatures, and decouple order status updates strictly from client browser redirects.

3. Administrative Access Leakage & Data Authorization (Requirements A9 & A10)

Risk: Administrative operations (product updates, order status changes) could be bypassed if authorization checks are only performed on the frontend UI layer rather than strictly on the backend.

Mitigation: Enforce role-based access control (RBAC) at the server level via custom Django middleware and view permissions (@user_passes_test / permission decorators) to block all unauthorized requests before executing business logic.